

SDA - Seminar - TAD Colecție

❖ Cuprins:

- Convenții
- TAD Colecție – definire și specificare
- TAD Iterator – specificare
- Exemplificare reprezentare
- Implementare Python

Convenții

- Algoritmii vor fi descriși în **Pseudocod**
- Elementele din containere vor fi elemente generale/abstracte
 - **TElement**
 - Operație de atribuire: $e_1 \leftarrow e_2$
 - Verificarea egalității: $e_1 = e_2$
 - **TComparabil**
 - Pentru containere ordonate
 - Permite inclusiv aplicarea unor relații de ordine: $<$, $>$ etc.

TAD Colecție

Definire

- Spre deosebire de cazul mulțimii, **elementele nu sunt neapărat distincte**
- Ca și în cazul mulțimii, **ordinea elementelor este irelevantă**
 - Nu există poziții într-o colecție. Așadar, operațiile colecției nu primesc poziții ca parametri și nu returnează poziții.
 - Elementele nu sunt stocate în ordinea adăugării (cel puțin, nu avem această garanție)

De exemplu, dacă adăugăm într-o colecție elementele 1,3,2,6,2,5,2, la afișarea conținutului colecției i, orice ordine a elementelor e posibilă:

o 1,2,2,2,3,6,5

o 1,3,2,6,2,5,2

o 1,5,6,2,3,2,2

etc...

Specificare

1) Definirea domeniului:

$C = \{c \mid c \text{ este o colecție cu elemente de tip TElement}\}$

2) Specificarea interfeței:

creeaza(c)

pre: adevărat

post: $c \in C$, c este colecția vidă (sau $\dim(c)=0$)

distruge(c)

pre: $c \in C$

post: colecția c a fost distrusă

adauga(c,e)

pre: $c \in C$, $e \in \text{TElement}$

post: $c' \in C$, $c' = c \oplus \{e\}$

sterge(c,e)

pre: $c \in C$, $e \in \text{TElement}$

post: $c' \in C$, $c' = c \setminus \{e\}$ (sau $c' = c \ominus \{e\}$); **OBS:** se șterge o singură apariție a elementului e)

$$\text{sterge} \leftarrow \begin{cases} \text{adevărat, dacă } e \in c \\ \text{fals, altfel} \end{cases}$$

cauta(c,e)

pre: $c \in C$, $e \in \text{TElement}$

post:

$$\text{cauta} \leftarrow \begin{cases} \text{adevărat, dacă } e \in c \\ \text{fals, altfel} \end{cases}$$

nrAparitii(c, e)

pre: $c \in C$, $e \in \text{TElement}$

post: $\text{nrAparitii} \leftarrow$ numărul de apariții ale lui e în c

dim(c)

pre: $c \in C$

post: $\text{dim} \leftarrow$ numărul total de elemente din c

iterator(c,i)

pre: $c \in C$

post: $i \in I$, i este iterator pe c și i referă un prim element din c dacă c este nevidă sau i este invalid, în caz contrar

TAD Iterator

Specificare

1) Definirea domeniului:

$I = \{i \mid i \text{ este iterator pe } c \in C\}$

2) Specificarea interfeței:

creeaza(i,c)

pre: $c \in C$

post: $i \in I$, i este iterator pe c și i referă un prim element din c dacă c este nevidă sau i este invalid, în caz contrar

valid(i)

pre: $i \in I$

post:

$\text{valid} \leftarrow \begin{cases} \text{adev\c{a}rat}, & \text{dac\c{a} elementul curent referit de } i \text{ este valid} \\ \text{fals}, & \text{altfel} \end{cases}$

urmator(i)

pre: $i \in I$, $\text{valid}(i)$

post: $i' \in I$, i' referă următorul element din colecția iterată față de cel referit de i dacă un astfel de element există sau este invalid, altfel

element(i, e)

pre: $i \in I$, $\text{valid}(i)$

post: $e \in \text{TElement}$, e este elementul curent referit de iterator

prim(i)

pre: $i \in I$

post: $\text{post}: i' \in I$, i este iterator pe c și i referă un prim element din c dacă c este nevidă sau i este invalid, în caz contrar

Exemplificare reprezentare:

O primă variantă de reprezentare (R1):

- Ca tablou unidimensional (vector) de elemente care se pot repeta
- În acest caz, iteratorul conține indexul elementului curent

Exemplu:

1	3	2	6	2	5	2
---	---	---	---	---	---	---

O a doua variantă de reprezentare (R2):

- Ca vector de perechi (element, frecvență), elementele din perechi fiind distincte
- În acest caz, **nu mai este suficient ca în reprezentarea iteratorului să avem doar indexul curent**. Este necesară și frecvența curentă pentru elementul de la indexul curent.

Exemplu:

(1, 1)	(3, 1)	(2, 3)	(6, 1)	(5, 1)
--------	--------	--------	--------	--------

Implementare

- a se vedea fișierele **R1.py** (prima reprezentare) și **R2.py** (a doua reprezentare)

Materiale suplimentare:

- Python-style iterators: <https://www.geeksforgeeks.org/python/iterators-in-python/>
- C++ iterators: <https://www.geeksforgeeks.org/cpp/iterators-c-stl/>